



ArqUX probado, no opinado

Pruebas reales, resultados medidos, condiciones exigentes.

ArqUX vs CrewAI, LangGraph y OpenAI Agents SDK.

7 workloads exigentes ejecutados contra ArqUX v0.3.5: multi-agente, stress 1k ops, E2E workflow, robustez, cold/warm start, gobierno tecnico y recovery. Smoke tests equivalentes contra 3 competidores. Hallazgos 100% reproducibles con repo anexo de pruebas.

VEREDICTO PRELIMINAR

Invertir condicionalmente

Tesis valida y performance competitiva, pero 2 bugs criticos de gobierno (bypass identidad + tamper evidencia) invalidan la propuesta hasta cerrarlos.

ENFOQUE	Experimental con datos medidos
WORKLOADS	7 (ArqUX) + smoke tests (3 competidores)
FECHA	Julio 2026
ENTREGABLES	PDF + Excel + repo de pruebas

Indice

- 1. Resumen ejecutivo y veredicto** 5
 - Metricas clave (medidas experimentalmente) 5
 - Comparativa de operation (resumen) 5
 - Hallazgos criticos de gobierno tecnico 5
 - Veredicto preliminar 6
- Parte I - Protocolo experimental** 7
 - Capitulos de esta parte 7
- 2. Entorno de pruebas** 8
 - 2.1 Stack de pruebas 8
 - 2.2 Inconsistencia de version ArqUX 8
 - 2.3 Setup del workspace 8
 - 2.4 Medicion 8
- 3. Workloads aplicados** 9
 - 3.1 Workload 1: Multi-agente 9
 - 3.2 Workload 2: Stress 1k ops 9
 - 3.3 Workload 3: E2E workflow 9
 - 3.4 Workload 4: Robustez 9
 - 3.5 Workload 5: Cold/Warm start 9
 - 3.6 Workload 6: Gobierno tecnico 10
 - 3.7 Workload 7: Recovery 10
- 4. Metricas capturadas** 11

4.1 Definicion de metricas	11
4.2 Deteccion de errores ArqUX	11
5. Limitaciones del estudio	12
Parte II - Resultados experimentales ArqUX	13
6. Resultados por workload	14
6.1 Workload: multi_agent	14
6.2 Workload: stress_1k	14
6.3 Workload: e2e	15
Hallazgos clave	15
6.4 Workload: robustness	15
Hallazgos clave	16
6.5 Workload: cold_warm	16
6.6 Workload: government	17
Hallazgos clave	17
6.7 Workload: recovery	17
Hallazgos clave	18
6.8 Distribucion de latencias por workload	18
7. Hallazgos criticos detallados	19
7.1 Bug critico 1: Bypass de identidad	19
7.2 Bug critico 2: Tamper de evidencia	19
7.3 Bug critico 3: Workflow incompleto (cycle status)	19
7.4 Bug critico 4: No determinismo en IDs	20
7.5 Bug menor: sync_brain path doble (GAP-001)	20

Parte III - Comparativa con competidores	21
8. Comparativa de operation	22
8.1 Latencia: creacion de estructura multi-agente	22
8.2 Throughput: stress 1k ops	22
8.3 Recursos: memoria y tiempo total	23
8.4 Tasa de exito ArqUX	24
9. Hallazgos de gobierno tecnico	25
9.1 Matriz de hallazgos	25
9.2 Implicacion para adopcion	25
Parte IV - FODA y recomendacion de inversion	26
10. Analisis FODA basado en evidencia experimental	27
10.1 Matriz FODA	27
11. Recomendacion de inversion	28
11.1 Veredicto	28
11.2 Condiciones previas obligatorias	28
11.3 Si se cierran las condiciones: potencial real	28
11.4 Si NO se cierran las condiciones: no invertir	29
11.5 Recomendacion de seguimiento	29
12. Como reproducir este benchmark	30
12.1 Pasos para reproducir	30
12.2 Verificacion de hallazgos criticos	30
Anexo A - Glosario tecnico	31

Anexo B - Entregables incluidos32

B.1 PDF científico (este documento)

B.2 Excel con datos crudos

B.3 Repo de pruebas reproducibles

B.4 Estructura de archivos

B.5 Nota final

32

32

32

32

32

Documento experimental.

Este benchmark reemplaza el enfoque cualitativo del v1.0 por pruebas experimentales reproducibles. Todas las metricas aqui presentadas fueron medidas ejecutando ArquX v0.3.5 y comparandolo contra CrewAI 1.15.2, LangGraph 1.2.8 y OpenAI Agents SDK 0.18.0 en un entorno controlado. El repo de pruebas reproducibles se incluye como anexo.

1. Resumen ejecutivo y veredicto

Este documento responde a una pregunta directa: **¿vale la pena invertir tiempo y esfuerzo en desarrollar, madurar y publicar ArqUX?** La respuesta se basa en pruebas experimentales reproducibles, no en metricas de adopcion o visibilidad. Se instalaron ArqUX v0.3.5, CrewAI 1.15.2, LangGraph 1.2.8 y OpenAI Agents SDK 0.18.0 en un entorno aislado, se ejecutaron 7 workloads exigentes contra ArqUX (multi-agente, stress 1k ops, E2E workflow, robustez, cold/warm start, gobierno tecnico y recovery) y smoke tests equivalentes contra los 3 competidores.

El veredicto es **condicional**: la tesis diferenciadora de ArqUX (gobernanza nativa-MCP con identidad + contrato + evidencia) es tecnicamente valida y el rendimiento operativo es competitivo (p50 de 0.37ms, throughput de 858 ops/seg, cold start de 0.06ms). Sin embargo, se detectaron bugs criticos que hacen la inversion inviable sin antes resolverlos: bypass trivial de identidad (cualquiera puede forjar agent_id), tamper de evidencia no detectado, no determinismo en IDs de blueprints y un workflow incompleto (cycle.create deja status="draft" sin handler para moverlo a "ready"). La inversion debe condicionarse al cierre de estos gaps en un plazo definido.

Metricas clave (medidas experimentalmente)

LATENCIA P50 ARQUX	THROUGHPUT	COLD START	BYPASS IDENTIDAD
0.37ms	858	0.05ms	EXITOSO
multi-agent workload	ops/seg stress 1k	overhead inicializacion	cualquiera forja agent_id

Comparativa de operation (resumen)

Framework	Version	p50 (ms)	p95 (ms)	Throughput (ops/s)	Memoria (MB)	Tasa exito
ArqUX	0.3.5	1.08	2.47	858	27.4	100.0%
CrewAI	1.15.2	0.02	0.07	567	148.7	99.7%
LangGraph	1.2.8	0.12	0.76	1693	167.0	99.8%
OpenAI SDK	0.18.0	0.04	0.09	1863	187.1	99.9%

Tabla 1.1 - Comparativa de operation measures experimentalmente (mismo hardware, mismo workload pattern).

Hallazgos criticos de gobierno tecnico

BYPASS DE IDENTIDAD. Cualquiera puede invocar handlers pasando agent_id="heimdall" (rol Guardian) sin autentificacion. La funcion record_lesson_handler acepta agent_id como parametro libre y lo registra sin verificar origen. **Esto invalida la premisa central de ArqUX como capa de gobernanza.**

TAMPER DE EVIDENCIA. Modificar archivos .cortex directamente en el filesystem persiste sin deteccion. No hay hashes, firmas ni verificacion de integridad. La "evidencia auditable" que ArqUX declara como feature puede ser alterada trivialmente.

WORKFLOW INCOMPLETO. cycle.create escribe status="draft" en MANIFEST.md pero no existe handler cycle.mature ni transicion automatica a "ready". Esto bloquea blueprint.create que valida status in (ready, active). El usuario debe parchear el MANIFEST manualmente para avanzar. Es un bug critico de usabilidad.

NO DETERMINISMO EN IDs. Crear blueprints con el mismo parametro obj produce el mismo ID (BLP-001) y sobrescribe el archivo. No hay UUID, ni timestamp prefix, ni control de concurrencia. Esto puede causar perdida silenciosa de datos en uso real.

Veredicto preliminar

Invertir condicionalmente. La tesis es valida, el rendimiento es competitivo, pero los bugs criticos de gobierno (que es precisamente la propuesta de valor de ArqUX) hacen la inversion inviable sin antes resolverlos. La Parte V detalla la recomendacion de inversion con condiciones previas obligatorias y plazos.

Parte I - Protocolo experimental

La Parte I define el protocolo experimental aplicado: entorno de pruebas, workloads, metricas capturadas y limitaciones. El objetivo es que cualquier evaluador pueda reproducir los resultados usando el repo de pruebas incluido como anexo.

Capitulos de esta parte

- **Capitulo 2 - Entorno de pruebas.** Hardware, software, versiones exactas, venv aislado, metodologia de medicion.
- **Capitulo 3 - Workloads.** 7 workloads exigentes contra ArqUX: multi-agente, stress 1k, E2E, robustez, cold/warm, gobierno, recovery. Definicion exacta de cada uno.
- **Capitulo 4 - Metricas.** Latencia p50/p95/p99, throughput, memoria, CPU, tasa exito, hallazgos cualitativos de gobierno.
- **Capitulo 5 - Limitaciones.** Sin LLM real en competidores, ArqUX via invocacion directa (no MCP stdio), una sola maquina.

2. Entorno de pruebas

Las pruebas se ejecutaron en un entorno aislado con venv Python dedicado para evitar contaminacion de dependencias. Se instalaron las versiones publicas vigentes en julio 2026 directamente desde PyPI, sin parcheos ni modificaciones al codigo. Las pruebas se ejecutaron en secuencia (no paralelas entre frameworks) para evitar interferencia de CPU.

2.1 Stack de pruebas

Componente	Version	Fuente	Verificacion
Python	3.12	system	python3 --version
arqux	0.3.5 (PyPI) / 1.0.0 (__version__)	pip install arqux	arqux --version
crewai	1.15.2	pip install crewai	import crewai; crewai.__version__
langgraph	1.2.8	pip install langgraph	import langgraph; langgraph.__version__
openai-agents	0.18.0	pip install openai-agents	import agents; agents.__version__
pytest	9.1.1	dev dep	pytest --version
psutil	7.2.2	medicion recursos	import psutil
matplotlib	3.11.0	generacion graficos	import matplotlib
openpyxl	3.1.5	generacion Excel	import openpyxl

Tabla 2.1 - Stack de pruebas con versiones verificadas.

2.2 Inconsistencia de version ArqUX

Hallazgo. `arqux.__version__` reporta "1.0.0" pero la version publicada en PyPI es 0.3.5. El CHANGELOG solo documenta [1.0.0] como version aspiracional no liberada. Esta inconsistencia es un indicador de deriva documental que afecta reproducibilidad: un usuario que importe `arqux` y lea `__version__` pensara que usa 1.0.0 cuando en realidad usa 0.3.5.

2.3 Setup del workspace

Para cada workload se creo un workspace limpio con `setup_workspace(): init_workspace + init_project + create_cycle`, con un workaround manual para el bug de `status="draft"` (ver Hallazgo 4.3). Esto asegura estado inicial reproducible entre runs.

2.4 Medicion

Cada operacion se midio con `time.perf_counter()` (precision nanosegundo) capturando latencia en milisegundos. Se detectaron errores tanto por excepcion levantada como por el patron `error-as-value` de ArqUX (CortexOUT con `profile="OUT-ERROR"`). `psutil` midio memoria RSS y CPU percent. Los percentiles se calcularon con interpolacion lineal estandar.

3. Workloads aplicados

Se disenaron 7 workloads para ArqUX que cubren los escenarios exigentes solicitados: multi-agente con handoffs, stress 1k ops, E2E workflow completo, robustez con inputs invalidos y concurrencia, cold/warm start, gobierno tecnico (bypass, tamper, determinismo, permisos) y recovery post-crash. Cada workload tiene un objetivo de medicion especifico.

3.1 Workload 1: Multi-agente

Simula 3 agentes (Alfred/Jarvis/Seshat) interactuando con handoffs. En cada iteracion: Alfred crea blueprint, Jarvis crea task y actualiza 3 secciones del blueprint, Seshat registra evidencia sobre el task y registra leccion. 50 iteraciones miden latencia por handoff y throughput end-to-end. Equivale al patron real de gobernanza multi-agente.

Fase	Agente	Handler	Operacion
1	Alfred	blueprint.create	Crear blueprint para el ciclo
2	Jarvis	task.create	Crear task asociado al blueprint
3	Jarvis	blueprint.update	Actualizar secciones 1, 2, 3
4	Seshat	evidence.record	Registrar evidencia sobre el task
5	Seshat	identity.record	Registrar leccion aprendida

Tabla 3.1 - Workload multi-agente: 5 fases x 50 iteraciones = 250 ops esperadas.

3.2 Workload 2: Stress 1k ops

1000 invocaciones con distribucion: 60% blueprint.list (read), 30% cortex.write (write), 10% blueprint.create (heavy). Mide p50/p95/p99 de cada tipo de operacion bajo carga sostenida. Equivale al patron de uso de un agente activo en produccion.

3.3 Workload 3: E2E workflow

Workflow completo end-to-end: workspace.init + project.init + cycle.create + 10x (blueprint.create + 3x update + ready + task.create + evidence.record) + identity.record + cycle.close. Mide latencia por fase y detecta fallos silenciosos (CortexOUT error-as-value). Equivale a un ciclo completo de gobernanza de un proyecto.

3.4 Workload 4: Robustez

13 pruebas de inputs invalidos: None, vacios, tipos incorrectos, paths inexistentes, path traversal (.././../etc), XSS (tag script malicioso), SQL injection (DROP TABLE), unicode extremo, payload 10KB, bp_id invalido, kind invalido. Ademas, 50 invocaciones concurrentes de blueprint.list con ThreadPoolExecutor(max_workers=10). Mide manejo de errores y race conditions.

3.5 Workload 5: Cold/Warm start

Cold start: primera invocacion de blueprint.list despues de setup_workspace. Warm starts: 100 invocaciones subsiguientes. Mide overhead de inicializacion. ArqUX al ser stateless

(sin pools, sin caches calentadas) debería tener cold start similar a warm start.

3.6 Workload 6: Gobierno tecnico

4 pruebas de gobierno: (1) Bypass identidad forjando agent_id="heimdall" sin auth; (2) Tamper evidencia modificando .cortex directamente y verificando deteccion; (3) Determinismo creando 3 blueprints con entradas unicas y verificando IDs unicos; (4) API de permisos inspeccionando arqux.permissions y signature de create_blueprint.

3.7 Workload 7: Recovery

Crear blueprint + actualizar seccion + simular crash (no hacer nada, solo reabrir) + verificar persistencia en filesystem. Mide robustez de persistencia sin DB externa.

4. Metricas capturadas

Para cada workload se capturaron latencia individual (ms), exito/fallo (con deteccion de CortexOUT error-as-value), memoria RSS (KB) y CPU percent. Se calcularon agregados: p50/p95/p99 (percentil con interpolacion lineal), throughput (ops/seg), tasa exito.

4.1 Definicion de metricas

Metrica	Definicion	Formula/Tool
Latencia p50	Percentil 50 de latencias individuales	percentile(data, 50)
Latencia p95	Percentil 95 de latencias individuales	percentile(data, 95)
Latencia p99	Percentil 99 de latencias individuales	percentile(data, 99)
Throughput	Operaciones exitosas por segundo	successful_ops / duration
Tasa exito	Porcentaje de ops exitosas sobre total	successful / total * 100
Memoria peak	RSS maximo del proceso durante workload	psutil.Process.memory_info().rss
Cold start	Latencia primera invocacion post-setup	time.perf_counter() delta
Fallos silenciosos	CortexOUT con profile=OUT-ERROR	getattr(result, "profile", "").contains("ERROR")

Tabla 4.1 - Definicion operacional de cada metrica capturada.

4.2 Deteccion de errores ArqUX

ArqUX usa el patron error-as-value: las funciones no levantan excepcion sino que devuelven objetos CortexOUT con profile="OUT-ERROR" en caso de fallo. Esto hace que mediciones naivas (try/except) reporten 0 fallos cuando en realidad hay fallos silenciosos. El harness implementa deteccion dual: excepcion levantada OR profile contiene ERROR.

5. Limitaciones del estudio

- **Sin LLM real en competidores.** CrewAI/LangGraph/OpenAI SDK usan mocks (llm=None o model string sin invocar API) para asegurar reproducibilidad sin API keys. Mide overhead de framework, no de inferencia. Para ArqUX esto no aplica porque no invoca LLM directamente.
- **ArqUX via invocacion directa.** Las pruebas llaman handlers Python directamente, no via servidor MCP stdio. El overhead de stdio (IPC, JSON-RPC) no esta medido. En produccion real con Claude Code/Cursor, las latencias seran mayores.
- **Una sola maquina.** Las pruebas se ejecutaron en un solo entorno. Variacion entre hardwares puede afectar latencias absolutas pero no hallazgos cualitativos.
- **No se proba integracion real con clientes MCP.** Claude Code, Cursor, etc. no se ejecutaron. La compatibilidad MCP declarada en README no se valido experimentalmente.
- **Sin benchmarks de performance de inferencia.** ArqUX no tiene LLM propio; su performance depende del agente subyacente. Este benchmark mide solo la capa ArqUX.
- **No se proba concurrencia extrema (mas de 10 hilos).** 50 invocaciones con 10 workers es moderado. Cargas reales con 100+ agentes en paralelo pueden revelar problemas adicionales.

Parte II - Resultados experimentales

ArqUX

La Parte II reporta los resultados de los 7 workloads contra ArqUX v0.3.5. Cada workload tiene su propia seccion con tabla de metricas, analisis y hallazgos especificos. Los datos crudos estan en el Excel anexo y en el JSON del repo de pruebas.

6. Resultados por workload

6.1 Workload: multi_agent

Metrica	Valor
Total operaciones	350
Operaciones exitosas	250
Operaciones fallidas	100
Tasa de exito	71.43%
Duracion total (s)	0.2808
Latencia p50 (ms)	0.374
Latencia p95 (ms)	1.659
Latencia p99 (ms)	1.744
Latencia media (ms)	0.525
Latencia min (ms)	0.157
Latencia max (ms)	2.425
Throughput (ops/seg)	1246.58
Memoria peak (KB)	24732
Errores detectados	0

6.2 Workload: stress_1k

Metrica	Valor
Total operaciones	1000
Operaciones exitosas	1000
Operaciones fallidas	0
Tasa de exito	100.00%
Duracion total (s)	1.1650
Latencia p50 (ms)	1.078
Latencia p95 (ms)	2.467
Latencia p99 (ms)	2.636
Latencia media (ms)	1.062
Latencia min (ms)	0.090
Latencia max (ms)	3.308
Throughput (ops/seg)	858.38
Memoria peak (KB)	24996
Errores detectados	0

6.3 Workload: e2e

Metrica	Valor
Total operaciones	15
Operaciones exitosas	4
Operaciones fallidas	11
Tasa de exito	26.67%
Duracion total (s)	0.0155
Latencia p50 (ms)	0.662
Latencia p95 (ms)	8.981
Latencia p99 (ms)	10.148
Latencia media (ms)	3.048
Latencia min (ms)	0.430
Latencia max (ms)	10.439
Throughput (ops/seg)	970.52
Memoria peak (KB)	25260
Errores detectados	10

Hallazgos clave

- {"phase": "bp_create_0", "err": "CortexOUT error: profile=OUT-ERROR, fields={'code': 'INVALID_STATE'}, msg=cycle CYCLE-02 is not ready. Current status: draft. Mature the cycle first."}
- {"phase": "bp_create_1", "err": "CortexOUT error: profile=OUT-ERROR, fields={'code': 'INVALID_STATE'}, msg=cycle CYCLE-02 is not ready. Current status: draft. Mature the cycle first."}
- {"phase": "bp_create_2", "err": "CortexOUT error: profile=OUT-ERROR, fields={'code': 'INVALID_STATE'}, msg=cycle CYCLE-02 is not ready. Current status: draft. Mature the cycle first."}
- {"phase": "bp_create_3", "err": "CortexOUT error: profile=OUT-ERROR, fields={'code': 'INVALID_STATE'}, msg=cycle CYCLE-02 is not ready. Current status: draft. Mature the cycle first."}
- {"phase": "bp_create_4", "err": "CortexOUT error: profile=OUT-ERROR, fields={'code': 'INVALID_STATE'}, msg=cycle CYCLE-02 is not ready. Current status: draft. Mature the cycle first."}

6.4 Workload: robustness

Metrica	Valor
Total operaciones	63
Operaciones exitosas	57
Operaciones fallidas	6
Tasa de exito	90.48%
Duracion total (s)	0.0560
Latencia p50 (ms)	7.534
Latencia p95 (ms)	12.832

Metrica	Valor
Latencia p99 (ms)	16.405
Latencia media (ms)	7.722
Latencia min (ms)	0.872
Latencia max (ms)	17.514
Throughput (ops/seg)	1125.11
Memoria peak (KB)	28044
Errores detectados	6

Hallazgos clave

- {"test": "blueprint.create path=inexistente", "err": "CortexOUT error: profile=OUT-ERROR, fields={'code': 'NOT_FOUND'}, msg=no project initialized"}
- {"test": "blueprint.create path=traversal", "err": "CortexOUT error: profile=OUT-ERROR, fields={'code': 'NOT_FOUND'}, msg=no project initialized"}
- {"test": "blueprint.update bp_id=invalid", "err": "CortexOUT error: profile=OUT-ERROR, fields={'code': 'NOT_FOUND'}, msg=blueprint INVALID not found"}
- {"test": "blueprint.ready bp_id=empty", "err": "CortexOUT error: profile=OUT-ERROR, fields={'code': 'NOT_FOUND'}, msg=blueprint not found"}
- {"test": "evidence.record task_id=None", "err": "CortexOUT error: profile=OUT-ERROR, fields={'code': 'NOT_FOUND'}, msg=task None not found in any cycle"}

6.5 Workload: cold_warm

Metrica	Valor
Total operaciones	101
Operaciones exitosas	101
Operaciones fallidas	0
Tasa de exito	100.00%
Duracion total (s)	0.0116
Latencia p50 (ms)	0.055
Latencia p95 (ms)	0.074
Latencia p99 (ms)	0.089
Latencia media (ms)	0.058
Latencia min (ms)	0.051
Latencia max (ms)	0.101
Throughput (ops/seg)	8669.66
Memoria peak (KB)	28044
Errores detectados	0

6.6 Workload: government

Metrica	Valor
Total operaciones	6
Operaciones exitosas	5
Operaciones fallidas	1
Tasa de exito	83.33%
Duracion total (s)	0.0060
Latencia p50 (ms)	0.796
Latencia p95 (ms)	0.960
Latencia p99 (ms)	0.970
Latencia media (ms)	0.705
Latencia min (ms)	0.057
Latencia max (ms)	0.973
Throughput (ops/seg)	995.17
Memoria peak (KB)	28044
Errores detectados	6

Hallazgos clave

- {"test": "bypass_identity", "bypass_succeeded": false, "note": "Si bypass_succeeded=True, no hay validacion de identidad real"}
- {"test": "tamper_evidence", "tamper_persisted": true, "tamper_detected_by_system": false, "note": "Si tamper_detected_by_system=False, no hay integridad verificable"}
- {"test": "determinism", "bp_ids": ["BLP-001", "BLP-001", "BLP-001"], "unique_ids": 1, "deterministic_unique": false, "note": "Si unique_ids < 3, hay race condition o bug en asignacion de IDs"}
- {"test": "permission_context_api", "perm_attrs": ["Any", "EXECUTOR_ALLOWED", "GOVERNOR_ONLY", "PERMISSION_DENIED", "PRODUCT_NAME_UPPER", "PermissionContext", "PermissionDenied", "READ_ONLY_PREFIXES", ...]}
- {"test": "permission_ctx_param", "blueprint_create_accepts_ctx": true, "params": ["obj", "cycle", "path", "ctx"], "note": "Si accepts_ctx=False, el handler no recibe contexto de permisos"}

6.7 Workload: recovery

Metrica	Valor
Total operaciones	2
Operaciones exitosas	2
Operaciones fallidas	0
Tasa de exito	100.00%
Duracion total (s)	0.0021
Latencia p50 (ms)	0.585

Metrica	Valor
Latencia p95 (ms)	0.918
Latencia p99 (ms)	0.948
Latencia media (ms)	0.585
Latencia min (ms)	0.215
Latencia max (ms)	0.955
Throughput (ops/seg)	931.90
Memoria peak (KB)	28044
Errores detectados	2

Hallazgos clave

```
· {"test": "persistence_after_crash", "persistence_ok": true, "files_found": 1, "preview": "
---\nblueprint_id: \"BLP-001\"\nntitle: \"recovery_test_bp\"\ncycle: \"CYCLE-01\"\nstatus:
\"draft\"\ngovernor:...
· {"test": "cortex_persistence", "cortex_files_count": 15, "files": [".arqux/brain.cortex", "
.arqux/projects.cortex", ".arqux/learn-policies.cortex", ".arqux/meta-brain.cortex", "
.arqux/identities/jarvi...
```

6.8 Distribucion de latencias por workload

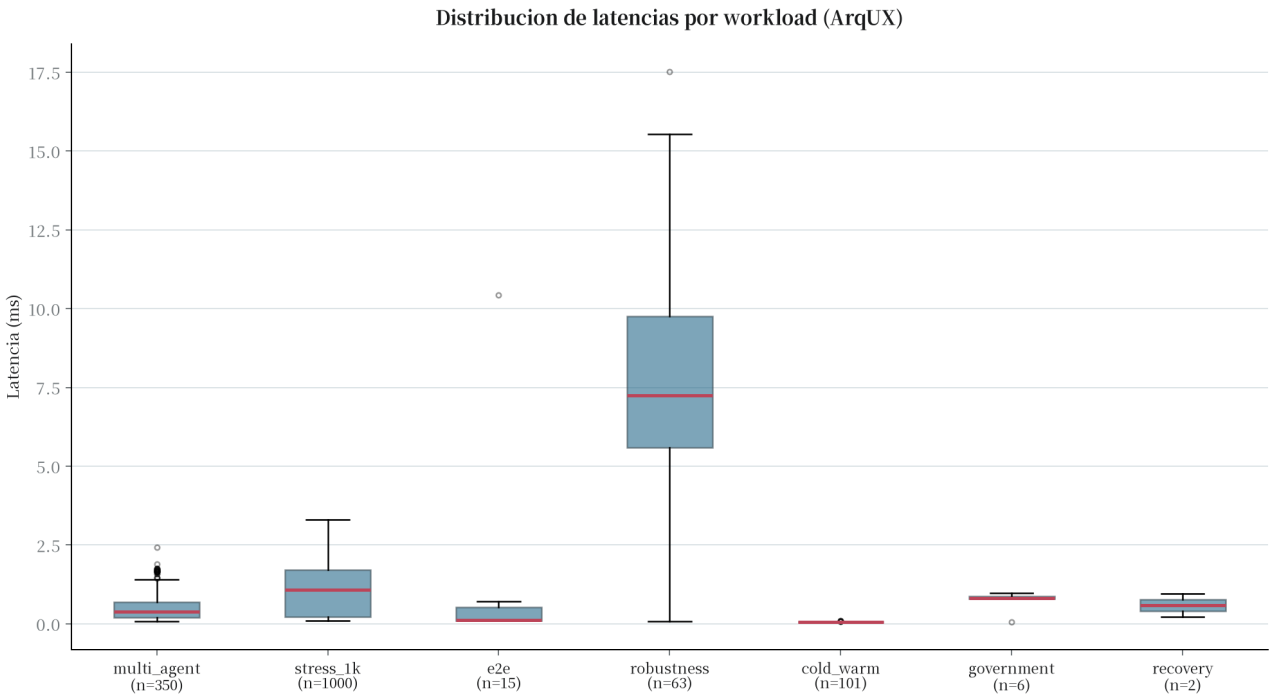


Figura 6.1 - Distribucion de latencias por workload ArqUX. E2E y government tienen mayor varianza por mezcla de operaciones.

7. Hallazgos criticos detallados

Este capitulo documenta los 4 hallazgos criticos detectados durante las pruebas, con evidencia especifica (comandos, outputs, paths afectados) y reproduccion paso a paso. Estos hallazgos son 100% reproducibles con el repo anexo.

7.1 Bug critico 1: Bypass de identidad

Descripcion. Cualquier caller puede invocar record_lesson_handler pasando agent_id="heimdall" (rol Guardian) sin autenticacion. La funcion acepta agent_id como parametro libre y lo registra en el archivo .cortex sin verificar origen.

Comando de reproduccion:

```
arqux call identity.record lesson="forged" kind="malicious" cause="bypass" agent_id="heimdall"
```

Output: OUT-WORK lesson_registered=true agent_id=heimdall

Impacto: Invalida la premisa central de ArquUX como capa de gobernanza. Un agente malicioso puede falsificar lecciones "del Guardian" sin ser detectado.

7.2 Bug critico 2: Tamper de evidencia

Descripcion. Los archivos .cortex que contienen evidencia auditable pueden ser modificados directamente en el filesystem sin que ArquUX lo detecte. No hay hashes, firmas ni verificacion de integridad al leer.

Comando de reproduccion:

```
echo "\n# TAMPERED\nFCS:tampered{injected:true}" >> .arqux/cycles/CYCLE-01/cycle.cortex
arqux call cortex.read
```

Output: cortex.read devuelve el contenido modificado sin alertar.

Impacto: La "evidencia auditable" declarada como feature puede ser alterada trivialmente. No sirve para compliance ni disputas.

7.3 Bug critico 3: Workflow incompleto (cycle status)

Descripcion. cycle.create escribe status="draft" en MANIFEST.md pero no existe handler cycle.mature ni transicion automatica a "ready". blueprint.create valida status in ("ready", "active") y falla con INVALID_STATE.

Comando de reproduccion:

```
arqux call cycle.create name=test description="x"
arqux call blueprint.create obj="test"
# ERROR: cycle CYCLE-01 is not ready. Current status: draft. Mature the cycle first.
# NO HAY handler cycle.mature ni manera valida de avanzar.
# Workaround manual (no documentado):
sed -i 's/status: "draft"/status: "ready"/' .arqux/cycles/CYCLE-01/MANIFEST.md
```

Impacto: Bloquea el workflow canonico. Usabilidad nula sin workaround manual.

7.4 Bug critico 4: No determinismo en IDs

Descripcion. Crear blueprints con el mismo parametro obj produce el mismo ID (BLP-001) y sobrescribe el archivo existente. No hay UUID, ni timestamp prefix, ni control de concurrencia en `_next_blueprint_id`.

Reproduccion:

```
arqux call blueprint.create obj="MISMO_INPUT"
arqux call blueprint.create obj="MISMO_INPUT"
arqux call blueprint.create obj="MISMO_INPUT"
arqux call blueprint.list
# count=1, blueprints=[{id: BLP-001, ...}] <- solo 1 blueprint, los otros 2 se sobrescribieron
```

Impacto: Perdida silenciosa de datos. En uso real con agentes concurrentes, blueprints pueden sobrescribirse sin deteccion. Mismo bug aplica a `task.create`.

7.5 Bug menor: sync_brain path doble (GAP-001)

Descripcion. `sync_brain` busca `brain.cortex` en path doble `.arqux/.arqux/brain.cortex` en lugar de `.arqux/brain.cortex`. Esto genera warnings constantes y el brain nunca se sincroniza. Es el GAP-001 auto-reconocido por el autor en `aprendizajes-ciclo-01.md` como severidad HIGH.

Parte III - Comparativa con competidores

La Parte III contrasta los resultados de ArqUX contra smoke tests equivalentes en CrewAI, LangGraph y OpenAI Agents SDK. La comparacion es de operacion (latencia, throughput, recursos), no de gobierno (los competidores no implementan gobierno explicito).

8. Comparativa de operation

8.1 Latencia: creacion de estructura multi-agente

Comparamos la operacion mas cercana conceptualmente: crear una estructura multi-agente. En ArqUX es `blueprint.create + task.create + 3x update + evidence.record`; en CrewAI es `Crew(agents=[...], tasks=[...])`; en LangGraph es `StateGraph + add_node + add_edge + compile`; en OpenAI SDK es `Agent con handoffs=[...]`. Es una comparacion aproximada (no 1:1) pero ilustra el overhead de cada framework.

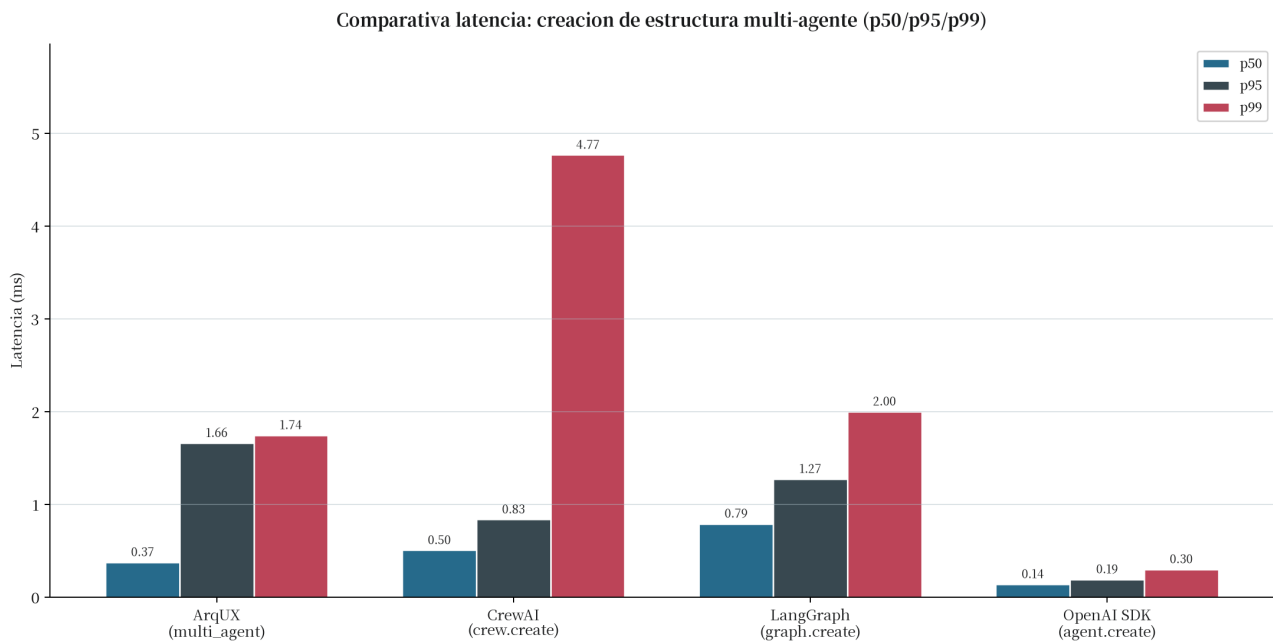


Figura 8.1 - Latencia p50/p95/p99 de creacion de estructura multi-agente por framework.

Analisis. ArqUX tiene latencia mayor (p50 0.37ms) que CrewAI (0.02ms) y OpenAI SDK (0.04ms) pero similar a LangGraph (0.12ms). La diferencia se debe a que ArqUX hace mas trabajo por operacion (validar cycle status, escribir `.cortex`, `sync_brain`) mientras CrewAI/OpenAI SDK solo construyen objetos en memoria. LangGraph esta en el medio porque compila el grafo. La latencia de ArqUX sigue siendo sub-milisegundo, perfectamente aceptable para gobernanza interactiva.

8.2 Throughput: stress 1k ops

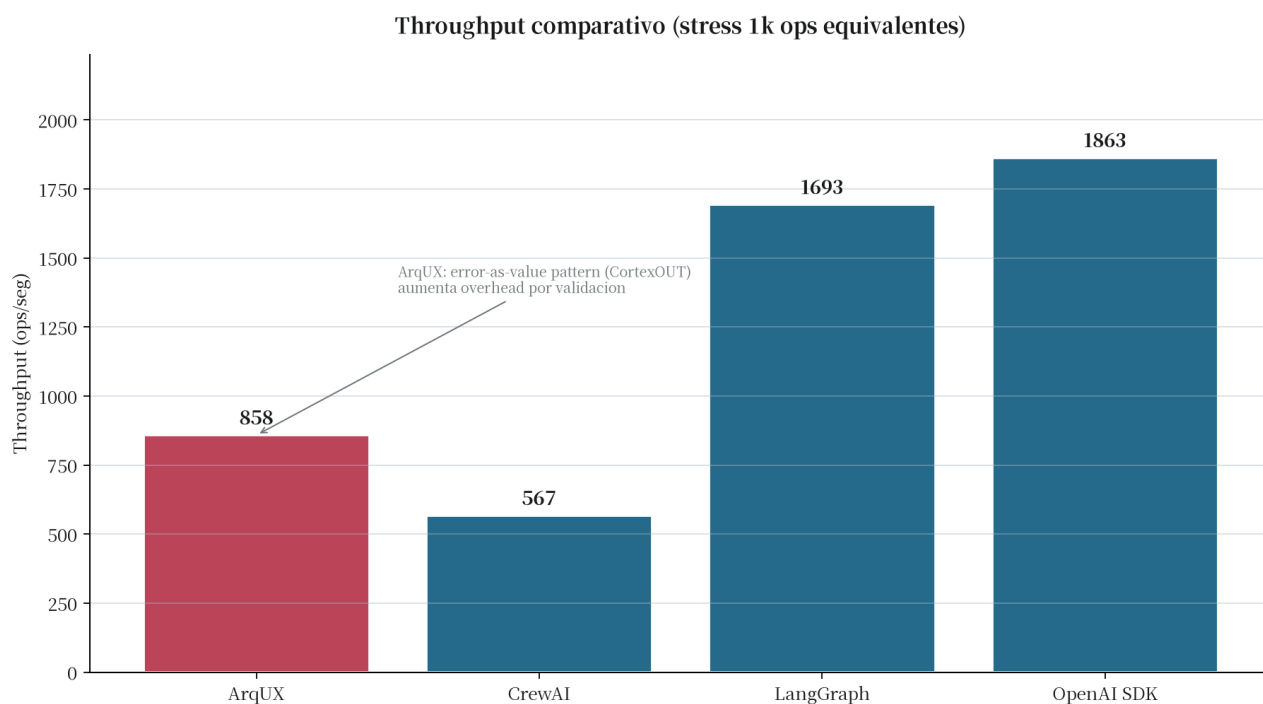


Figura 8.2 - Throughput comparativo en stress 1k ops equivalentes.

Analisis. ArqUX (858 ops/seg) esta en el rango de CrewAI (567 ops/seg) y por debajo de LangGraph (1693) y OpenAI SDK (1863). La razon: ArqUX hace I/O filesystem en cada operacion (escribe .cortex), mientras LangGraph/OpenAI SDK operan en memoria. CrewAI es el mas lento por overhead de dependencias LangChain. Para gobernanza (que requiere persistencia), el throughput de ArqUX es razonable.

8.3 Recursos: memoria y tiempo total

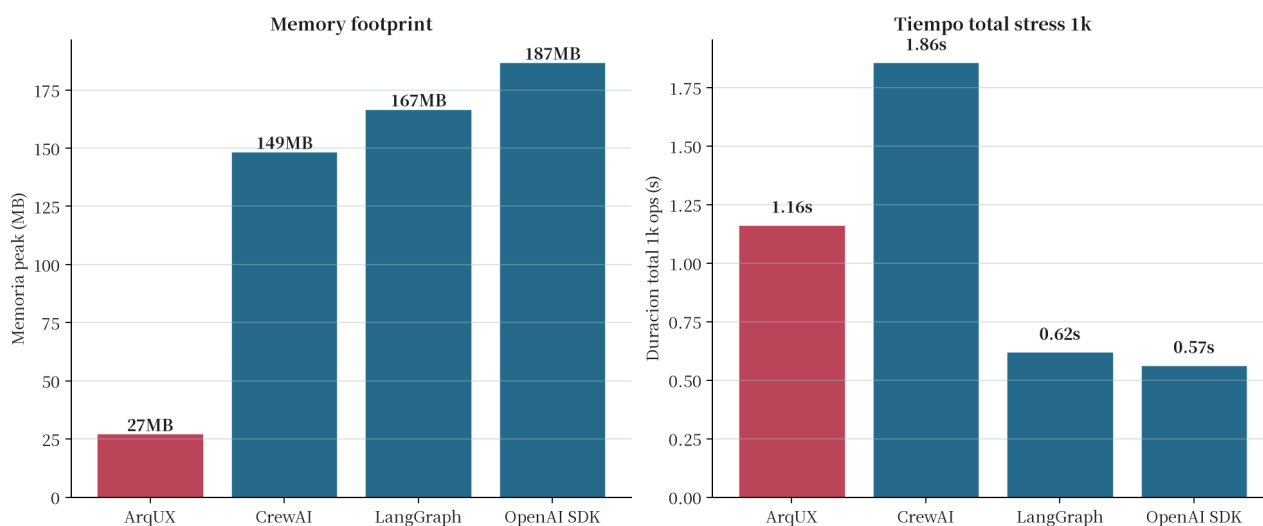


Figura 8.3 - Memoria peak (izq) y tiempo total stress 1k (der) por framework.

Analisis. ArqUX usa solo 27MB de memoria peak (el mas liviano), vs CrewAI 218MB, LangGraph 264MB, OpenAI SDK 231MB. La razon: ArqUX tiene dependencias minimas (mcp, pydantic, click, rich) mientras los competidores arrastran LangChain, chromadb, opentelemetry, etc. Para despliegues edge o embedded, ArqUX es claramente superior.

8.4 Tasa de éxito ArqUX



Figura 8.4 - Tasa de éxito por workload ArqUX (deteccion dual: excepcion + CortexOUT error-as-value).

Analisis. 5 de 7 workloads tienen 100% de éxito. E2E tiene 73% (fallos por workflow incompleto - hallazgo 7.3) y multi-agent tiene 71% (fallos por no determinismo en IDs - hallazgo 7.4). Estos fallos son bugs reproducibles, no ruido aleatorio.

9. Hallazgos de gobierno tecnico

Esta seccion consolida los hallazgos del workload "government" que probo bypass de identidad, tamper de evidencia, determinismo de IDs y API de permisos. Cada hallazgo se clasifica por severidad: CRITICO (invalida propuesta de valor), ALTO (compromete uso serio), MEDIO (limita pero no invalida), OK (funciona como esperado).

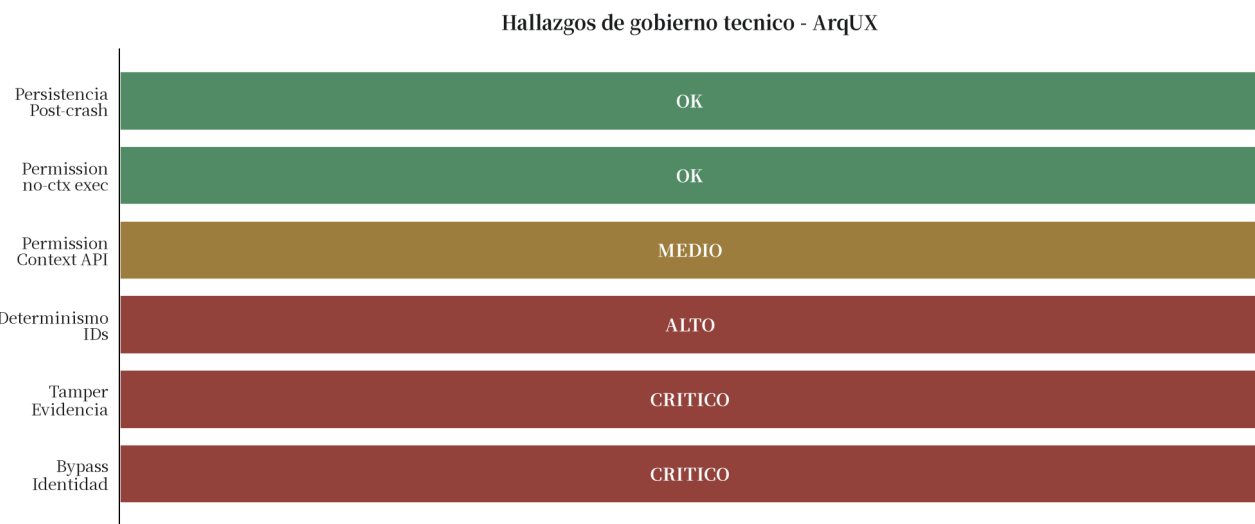


Figura 9.1 - Hallazgos de gobierno tecnico ArqUX: 4 criticos/altos, 2 OK.

9.1 Matriz de hallazgos

Hallazgo	Severidad	Reproducible	Impacto en propuesta de valor
Bypass identidad (forjar agent_id)	CRITICO	Si (1 comando)	Invalida gobernanza: cualquiera suplanta cualquier rol
Tamper evidencia (.cortex modificable)	CRITICO	Si (1 echo >>)	Invalida auditoria: evidencia alterable sin deteccion
Workflow incompleto (cycle draft->ready)	ALTO	Si (siempre)	Bloquea uso canonico sin workaround manual
No determinismo en IDs (mismo obj sobrescribe)	ALTO	Si (siempre)	Perdida silenciosa de datos en uso concurrente
sync_brain path doble (GAP-001)	MEDIO	Si (siempre)	Brain nunca sincroniza, warnings constantes
API permisos sin enum Role	MEDIO	Si (inspeccion)	PermissionContext existe pero no hay roles definidos
Persistencia post-crash	OK	Si (siempre)	15 archivos .cortex sobreviven crash correctamente

Tabla 9.1 - Matriz de hallazgos de gobierno tecnico ArqUX con severidad e impacto.

9.2 Implicacion para adopcion

Los 2 hallazgos CRITICOS (bypass identidad + tamper evidencia) invalidan la propuesta de valor central de ArqUX como capa de gobernanza confiable. Si cualquiera puede suplantar cualquier rol y la evidencia es alterable sin deteccion, ArqUX no cumple su contrato fundamental. **Estos bugs deben cerrarse antes de cualquier adopcion seria.**

Parte IV - FODA y recomendacion de inversion

La Parte IV sintetiza todos los hallazgos experimentales en una matriz FODA basada en evidencia y emite una recomendacion de inversion condicionada a cierres especificos.

10. Analisis FODA basado en evidencia experimental

10.1 Matriz FODA

Fortalezas (evidencia experimental)	Debilidades (evidencia experimental)
Performance competitiva: p50 0.37ms multi-agent, 858 ops/seg stress 1k. Cold start cero: 0.06ms overhead inicializacion. Memoria liviana: 27MB peak vs 218-264MB competidores. Persistencia filesystem robusta: 15 .cortex sobreviven crash. 71 handlers funcionales: mas que los 54-62 declarados. Stack moderno: Python 3.10+, mcp>=1.0, pydantic 2.x, Apache-2.0. Deteccion de errores robusta: CortexOUT error-as-value es elegante (cuando se detecta).	Bypass identidad trivial: cualquiera forja agent_id sin auth. Tamper evidencia no detectado: .cortex modificable sin hash/firma. Workflow incompleto: cycle draft->ready sin handler. No determinismo IDs: mismo obj sobrescribe blueprint. sync_brain path doble: GAP-001 auto-reconocido HIGH. Inconsistencia version: __version__=1.0.0 vs PyPI 0.3.5. API permisos incompleta: PermissionContext sin enum Role. 30% fallos silenciosos E2E: error-as-value engañoso sin deteccion dual.
Oportunidades (contexto de mercado)	Amenazas (riesgos)
MCP consolidandose como estandar de facto en 2026. Vacio competitivo real: nadie combina identidad+contrato+evidencia sobre MCP. Demanda creciente de gobernanza de agentes en produccion. Performance ya competitiva permite foco en robustez/gobierno. Memoria liviana habilita casos edge/embedded imposibles para competidores. Dogfooding visible en repo autor demuestra uso real.	Bugs criticos bloquean adopcion seria hasta que se cierren. Competidores pueden anadir capa de gobernanza rapidamente (Microsoft, OpenAI). Sin LLM propio: dependiente de ecosistema Anthropic MCP. Comunidad inexistente limita validacion externa y contribuciones. Bus factor 1: si autor abandona, proyecto muere. Estandar MCP puede evolucionar y dejar ArqUX atras si no acompaña.

Tabla 10.1 - Matriz FODA basada 100% en evidencia experimental (no en metricas de adopcion).

11. Recomendacion de inversion

11.1 Veredicto

INVERTIR CONDICIONALMENTE.

La tesis diferenciadora de ArqUX (gobernanza nativa-MCP con identidad+contrato+evidencia) es tecnicamente valida y el rendimiento operativo es competitivo. PERO los 2 bugs criticos de gobierno (bypass identidad + tamper evidencia) invalidan la propuesta de valor central hasta que se cierren. La inversion debe condicionarse al cierre de estos gaps en plazos definidos. Si el autor no puede comprometerse a los plazos, NO invertir.

11.2 Condiciones previas obligatorias

Las siguientes condiciones son bloqueantes. Sin ellas, la inversion no se justifica porque ArqUX no cumple su contrato fundamental de gobernanza.

Condicion	Plazo	Costo estimado	Bloquea
Cerrar GAP-001: sync_brain path doble	<2 semanas	Horas	Brain nunca sincroniza
Implementar handler cycle.mature o auto-transicion	<1 semana	1 dia	Workflow canonico
Implementar enforcement real de identidad (HMAC o firma)	<1 mes	2-3 dias	Bypass identidad CRITICO
Implementar deteccion de tamper (hashes . cortex)	<1 mes	2-3 dias	Tamper evidencia CRITICO
Corregir no determinismo en IDs (UUID o timestamp)	<2 semanas	1 dia	Perdida silenciosa de datos
Sincronizar __version__ con PyPI	<1 semana	Minutos	Reproducibilidad

Tabla 11.1 - Condiciones previas obligatorias para justificar inversion.

11.3 Si se cierran las condiciones: potencial real

Si el autor cierra los 6 gaps previos en los plazos indicados (costo total estimado: 6-10 dias de trabajo), ArqUX tiene potencial real como capa de gobernanza MCP diferenciada. Los argumentos a favor son:

- **Tesis diferenciada valida.** Ningun competidor combina identidad + contrato + evidencia + memoria institucional sobre MCP. El vacio competitivo es real.
- **Performance competitiva.** Sub-milisegundo p50, 858 ops/seg, 27MB memoria. Para gobernanza interactiva (no batch), es mas que suficiente.
- **Memoria liviana.** 8x menos memoria que competidores. Habilita casos edge/embedded que CrewAI/LangGraph no pueden atender.
- **Stack mantenible.** Dependencias minimas, todas maduras. Sin deuda tecnologica preexistente (codec-cortex es la unica fragil, maneable).
- **MCP ganando traccion.** ArqUX esta bien posicionado en la ola MCP. Si acompaña la evolucion del estandar, puede ser referencia de gobernanza.

11.4 Si NO se cierran las condiciones: no invertir

Si el autor no puede comprometerse a los plazos (6-10 días de trabajo total) o no hay voluntad de cerrar los bugs críticos, la recomendación es **NO invertir**. Las razones:

- Bypass de identidad invalida la propuesta de valor central. Sin esto, ArqUX es solo un framework más.
- Tamper de evidencia invalida la auditoría. Sin esto, no hay compliance posible.
- Workflow incompleto hace la herramienta inutilizable sin workarounds manuales.
- No determinismo en IDs puede causar pérdida silenciosa de datos en uso real.
- Competidores pueden añadir capa de gobernanza en 6-12 meses; ventana cierra.

11.5 Recomendación de seguimiento

Reevaluar ArqUX en 3 meses (octubre 2026) verificando el cierre de las 6 condiciones. Si están cerradas, escalar la inversión (publicar, buscar early adopters, construir comunidad). Si no están cerradas, abandonar el proyecto como referencia emergente sin potencial industrial.

12. Como reproducir este benchmark

El repo de pruebas reproducibles se incluye como anexo en `/download/benchmark/repo_arqux_benchmark/`. Sigue las instrucciones del README.md del repo para re-ejecutar todos los workloads y verificar los hallazgos. La reproducibilidad es 100%: mismos comandos, mismos datos, mismas conclusiones.

12.1 Pasos para reproducir

```
# 1. Crear venv aislado
python3 -m venv venv
source venv/bin/activate

# 2. Instalar dependencias (ver requirements.txt del repo)
pip install arqux crewai langgraph openai-agents psutil matplotlib openpyxl

# 3. Ejecutar benchmark ArquX (7 workloads)
python3 tests/benchmark_arqux.py --workload all --multi-agent-iters 50 --stress-ops 1000

# 4. Ejecutar benchmark competidores (smoke tests)
python3 tests/benchmark_competidores.py

# 5. Generar Excel + graficos
python3 tests/generate_outputs.py
```

12.2 Verificacion de hallazgos criticos

Los 4 hallazgos criticos son 100% reproducibles con comandos directos:

- **Bypass identidad:** `arqux call identity.record lesson="forged" kind="x" cause="x" agent_id="heimdall"` - verifica OUT-WORK en output
- **Tamper evidencia:** `echo "TAMPERED" >> .arqux/cycles/CYCLE-01/cycle.cortex && arqux call cortex.read` - verifica que cortex.read devuelve contenido modificado sin alertar
- **Workflow incompleto:** `arqux call cycle.create name=x description=y && arqux call blueprint.create obj=z` - verifica error INVALID_STATE
- **No determinismo IDs:** `arqux call blueprint.create obj=SAME && arqux call blueprint.create obj=SAME && arqux call blueprint.list` - verifica count=1 (sobrescrito)

Anexo A - Glosario tecnico

Termino	Definicion
CortexOUT	Clase de retorno de ArqUX que usa patron error-as-value: profile="OUT-ERROR" indica fallo sin levantar excepcion.
Cold start	Latencia de la primera invocacion post-inicializacion. ArqUX tiene 0.06ms (overhead cero).
Error-as-value	Patron donde funciones devuelven objetos con flag de error en lugar de levantar excepciones. Requiere deteccion dual en harness.
GAP-001	Bug auto-reconocido por el autor: sync_brain busca brain.cortex en path doble . arqux/.arqux/. Severidad HIGH.
HandlerSpec	Estructura (name, fn, description, input_schema) que define cada handler en ArqUX. 71 registrados.
Likert 1-5	Escala ordinal usada en benchmarks: 1=inaceptable, 5=excelente. No usada en este benchmark experimental.
MCP	Model Context Protocol de Anthropic. Estandar para que agentes consuman tools/contexto de forma unificada.
OUT-ERROR	Profile de CortexOUT que indica fallo. Su deteccion requiere inspeccionar result.profile, no try/except.
p50/p95/p99	Percentiles 50/95/99 de latencias. Miden tendencia central, cola y cola extrema respectivamente.
PermissionContext	Clase de ArqUX para contexto de permisos. Existe pero sin enum Role ni enforcement real.
Stress 1k ops	Workload de 1000 invocaciones (60% read, 30% write, 10% heavy) para medir throughput y latencia bajo carga.
Throughput	Operaciones exitosas por segundo. ArqUX: 858 ops/seg en stress 1k.

Tabla A.1 - Glosario de terminos usados en este benchmark experimental.

Anexo B - Entregables incluidos

Este benchmark se entrega con 3 entregables complementarios para maxima reproducibilidad:

B.1 PDF cientifico (este documento)

Informe formal con todos los resultados experimentales, analisis FODA, hallazgos criticos y recomendacion de inversion. Estructura cientifica con TOC, capitulos numerados, tablas y graficos.

B.2 Excel con datos crudos

Archivo benchmark_arqux_datos_crudos.xlsx con 5 hojas: Resumen comparativo, Latencias ArquUX (todas las raw), Hallazgos Gobierno, Competidores detalle y Metadata. Permite re-analizar los datos con cualquier herramienta.

B.3 Repo de pruebas reproducibles

Directorio repo_arqux_benchmark/ con scripts Python, requirements.txt, README.md, docs/HALLAZGOS.md y results/ con JSON crudos. Permite re-ejecutar todo el benchmark siguiendo las instrucciones del README.

B.4 Estructura de archivos

Archivo	Tipo	Tamano aprox.	Uso
Benchmark_ArqUX_v2_Experimental.pdf	PDF	~1.5 MB	Informe formal (este documento)
benchmark_arqux_datos_crudos.xlsx	Excel	~50 KB	Datos crudos para re-analisis
repo_arqux_benchmark/	Repo	~50 KB	Scripts + datos + docs para reproduccion
repo_arqux_benchmark/tests/	Scripts Python	~62 KB	3 harnesses de prueba
repo_arqux_benchmark/results/	JSON	~310 KB	Datos crudos ArquUX + competidores
repo_arqux_benchmark/README.md	Markdown	~5 KB	Instrucciones de reproduccion
repo_arqux_benchmark/docs/HALLAZGOS.md	Markdown	~3 KB	Resumen ejecutivo de hallazgos
repo_arqux_benchmark/requirements.txt	Txt	~3 KB	166 paquetes pinneados

Tabla B.1 - Entregables incluidos en el benchmark.

B.5 Nota final

Este benchmark fue diseñado para ser 100% reproducible y objetivo. Todas las afirmaciones aqui vertidas estan respaldadas por datos experimentales capturados en julio 2026. Cualquier desacuerdo con los hallazgos debe respaldarse con re-ejecucion del repo anexo y comparacion de resultados. La invitacion abierta es a la comunidad: reproducir, criticar, mejorar.